Qualidade de Sistemas e Software: Uma Abordagem Teórica e Prática sobre Métodos de Teste

Aristofenes Paulo Gonfulo

Monádio Celestina Tonta

Engenharia informática, Instituto Superior Politécnico Katangoji

Teste e Qualidade de Software

Prof. MSc. Mariel Morais

03 de novembro de 2026

Sumário

Objetivo Geral 4

Objetivos Específicos 4

Fundamentação Teórica 5

Conceitos Fundamentais de Sistemas e Software 5

Distinção entre Sistema e Software 5

Evolução do Conceito de Qualidade na Engenharia 6

Norma ISO/IEC 25010 e Qualidade 7

Testes de Software na Garantia da Qualidade 8

Estudo de Caso: Testes e Qualidade no Contexto Angolano 10

Conclusão 11

Referências 13

Qualidade de Sistemas e Software: Uma Abordagem Teórica e Prática sobre Métodos de Teste

Este trabalho analisa a qualidade de sistemas e software, iniciando pela etimologia dos conceitos para fundamentar a compreensão da temática. O vocábulo qualidade provém do latim *qualitas*, utilizado historicamente para indagar a natureza ou constituição essencial de um objeto (Dicionário Priberam da Língua Portuguesa, s.d.-a). Por sua vez, a palavra sistema deriva do grego *sýstēma*, que expressa a noção de elementos reunidos de forma coordenada e integrada (Dicionário Priberam da Língua Portuguesa, s.d.-b). A compreensão destas raízes evidencia que a qualidade não existe no vácuo; ela traduz-se no grau de adequação das características inerentes a um conjunto ordenado de componentes face às expectativas criadas.

No domínio da Engenharia de Software, essas definições adquirem contornos técnicos e operacionais rigorosos, tornando essencial distinguir sistema de software. Um sistema constitui uma infraestrutura abrangente composta por hardware, redes de comunicação, bases de dados, regras de negócio e atores humanos que interagem para cumprir um propósito global (Sommerville, 2019). Em contrapartida, o software representa o componente lógico dessa estrutura, congregando o código-fonte, algoritmos, rotinas de configuração e documentação que governam o comportamento computacional. Assim, falar de qualidade de sistemas e software exige uma visão holística: além de aferir a robustez estrutural do código de forma isolada, é mandatório avaliar como essa aplicação se comporta, comunica e responde dentro do ecossistema tecnológico mais amplo em que se encontra operacionalizada.

Para balizar esses critérios de excelência, a área apoia-se em modelos e normas internacionais, com destaque para a ISO/IEC 25010, que classifica a qualidade através de atributos essenciais como desempenho, fiabilidade, segurança, manutenibilidade e usabilidade (Pressman & Maxim, 2020). Todavia, a qualidade não se restringe a uma aspiração teórica; ela necessita de métodos práticos de comprovação. É nesse contexto que o teste de software assume um papel central como atividade empírica de verificação e

validação, orientada a identificar defeitos precocemente e assegurar que as especificações acordadas foram integralmente atendidas antes da entrega em ambiente de produção (Myers et al., 2012). Testar, portanto, consolida-se como o mecanismo primário de controlo que torna mensurável a garantia da qualidade ao longo de todo o ciclo de vida do desenvolvimento.

Diante do exposto, o presente trabalho académico tem como objetivo geral examinar os principais conceitos, normas e dimensões que caracterizam a qualidade de sistemas e software, destacando os processos de teste como ferramenta indispensável para a sua aferição. Para tanto, a pesquisa divide-se em fundamentação teórica dos atributos de qualidade, análise comparativa entre as camadas de sistema e produto de software, e discussão dos métodos de teste aplicáveis à garantia da conformidade em projetos computacionais.

Objetivo Geral

- Analisar os fundamentos conceituais, os modelos normativos (ISO/IEC 25010) e os processos práticos de teste aplicados à garantia da qualidade de sistemas e software, destacando a sua relevância na mitigação de falhas em soluções tecnológicas operacionais.

Objetivos Específicos

- **Diferenciar** tecnicamente sistema computacional de produto de software, delimitando as origens de falhas lógicas e estruturais de infraestrutura.
- **Examinar** as oito características de qualidade preconizadas pela norma ISO/IEC 25010 e a sua relevância para o ciclo de vida do desenvolvimento de sistemas.
- **Identificar** os níveis (unitário, integração, sistema e aceitação) e as tipologias de testes fundamentais na verificação e validação de requisitos funcionais e não funcionais.
- **Ilustrar** a aplicação prática dos conceitos de qualidade e técnicas de teste em cenários reais do contexto tecnológico nacional.

Fundamentação Teórica

Conceitos Fundamentais de Sistemas e Software

Distinção entre Sistema e Software

Para analisar a qualidade de maneira precisa na área de tecnologia, é indispensável estabelecer uma separação técnica entre o conceito de sistema computacional e o de produto de software. Embora muitas vezes utilizados como sinónimos no senso comum, estes dois termos representam níveis diferentes de abstração e complexidade dentro da engenharia.

Um produto de software é estritamente o componente lógico e imaterial da solução computacional. De acordo com Pressman e Maxim (2020), o software é formado pelo código-fonte dos programas, pelas rotinas algorítmicas, pelas estruturas de dados responsáveis pela manipulação da informação e pelos manuais ou documentações técnicas que orientam a sua operação. O software é concebido para resolver problemas específicos por meio de instruções que determinam como os dados devem ser processados, transformados e exibidos.

Em contraste, um sistema computacional compreende um ecossistema muito mais abrangente e heterogéneo. Segundo Sommerville (2019), um sistema é uma coleção de elementos interconectados e organizados para atingir um objetivo comum. Este ecossistema inclui não apenas o software em execução, mas também a infraestrutura de hardware (servidores, processadores, unidades de armazenamento), os elementos de conectividade e rede, as bases de dados e os processos de negócio da organização. Mais relevante ainda, o sistema inclui o elemento humano (os utilizadores e operadores) que interagem com a tecnologia no dia a dia. O software, portanto, não atua de forma autónoma no vácuo; ele é uma peça constituinte fundamental que dá vida e finalidade operacional ao sistema global.

Essa diferenciação tem impacto direto na abordagem de avaliação e garantia da qualidade. Um problema de execução num ambiente produtivo pode ter origens bastante distintas. Enquanto a falha de software decorre de um defeito lógico (*bug*), má estruturação de código ou erro de sintaxe cometido durante o ciclo de programação, uma falha de sistema

pode manifestar-se mesmo quando o código está formalmente correto. Por exemplo, congestionamentos de tráfego de rede, latência de servidores, memória RAM insuficiente ou erros operacionais introduzidos pelos utilizadores configuram falhas sistémicas que degradam o desempenho global (Sommerville, 2019). Por essa razão, a garantia da qualidade na engenharia exige que os testes verifiquem tanto o produto de software individualmente quanto a integridade do sistema computacional integrado.

Evolução do Conceito de Qualidade na Engenharia

A compreensão de qualidade na tecnologia não nasceu pronta; ela migrou da produção industrial para o mundo digital. Nas fábricas tradicionais, a qualidade resumia-se a inspecionar peças físicas no final de uma linha de montagem e descartar as que apresentassem defeitos visíveis (Garvin, 1988). No entanto, o software não é um objeto físico que se desgasta com o atrito ou que enferruja com o tempo: ele é puro pensamento lógico traduzido em código. Quando um sistema falha em operação, essa anomalia não surgiu espontaneamente pelo uso contínuo, mas resultou de uma falha de conceção introduzida silenciosamente durante a sua construção (Pressman & Maxim, 2020).

Compreender essa natureza imaterial forçou a engenharia a mudar a sua mentalidade de controlo. A qualidade deixou de significar "corrigir erros depois do produto pronto" e passou a significar "prevenir erros durante todo o processo de conceção". Essa transição fundamenta-se num princípio económico e estrutural implacável: o custo de reparar um defeito cresce exponencialmente à medida que o projeto avança. Descobrir e ajustar uma inconsistência ainda na fase de planeamento exige apenas rever especificações textuais; encontrar o mesmo erro após a aplicação estar em funcionamento operacional pode paralisar serviços essenciais, comprometer dados institucionais e multiplicar despesas financeiras em escalas críticas (Sommerville, 2019).

Por fim, a maturidade da disciplina revelou que a qualidade possui duas faces inseparáveis: a interna e a externa. A qualidade interna reflete a organização arquitetural, a clareza e a facilidade de manutenção do código-fonte, características invisíveis para o

utilizador comum, mas determinantes para os engenheiros que sustentam a solução. Em contrapartida, a qualidade externa traduz a experiência empírica de quem opera o sistema: rapidez de resposta, precisão das funcionalidades e facilidade de interação (ISO/IEC, 2011). Um sistema verdadeiramente maduro não escolhe entre uma e outra; ele une uma engenharia interna robusta a uma entrega final que resolve com fiabilidade a necessidade humana para a qual foi projetado.

Norma ISO/IEC 25010 e Qualidade

Para evitar que a qualidade seja avaliada com base em opiniões subjetivas ou impressões individuais, a comunidade técnica internacional convergiu para um padrão rigoroso e mensurável: a família de normas SQuaRE (*Software Product Quality Requirements and Evaluation*), corporizada principalmente na norma ISO/IEC 25010 (ISO/IEC, 2011). Essa norma funciona como um mapa detalhado da anatomia de um sistema, decompondo a noção ampla de qualidade em oito características fundamentais que determinam se uma solução tecnológica está apta para operar no mundo real.

A primeira dimensão é a **adequação funcional**, que responde à pergunta mais elementar do desenvolvimento: o software faz exatamente o que foi concebido para fazer? Esta dimensão mede a capacidade do produto em entregar todas as funções prometidas com precisão numérica e conformidade às regras de negócio definidas (Pressman & Maxim, 2020). De nada serve uma arquitetura sofisticada se as operações básicas não gerarem os resultados corretos esperados pelo utilizador.

A segunda e a terceira dimensões dizem respeito ao comportamento prático da aplicação: a **eficiência de desempenho** e a **fiabilidade**. A eficiência avalia a rapidez e o custo computacional com que as tarefas são executadas, assegurando que o sistema responde com tempos de espera reduzidos sem desperdiçar memória ou capacidade de processamento. Já a fiabilidade avalia a resiliência do sistema perante imprevistos. Um software fiável é aquele que não entra em colapso súbito perante variações de carga, mantém

os seus serviços ativos de forma contínua e consegue restaurar os seus dados intactos em caso de interrupção inesperada (Sommerville, 2019).

A vertente relacional do sistema divide-se entre as pessoas e a infraestrutura, expressa na **usabilidade**, na **segurança** e na **compatibilidade**. A usabilidade dita a facilidade com que um ser humano aprende, navega e domina a ferramenta com o menor esforço mental possível. A segurança ergue-se como a proteção dos ativos digitais, garantindo que apenas agentes autorizados acedam aos dados e impedindo manipulações maliciosas. Por sua vez, a compatibilidade reflete a capacidade de convivência tecnológica: mede a prontidão do software para coexistir e trocar informações de forma fluida com outros sistemas e serviços sem causar interferências mútuas (ISO/IEC, 2011).

Por fim, a norma consagra a longevidade da solução através da **manutenibilidade** e da **portabilidade**. A manutenibilidade mede o esforço futuro: quando as regras de negócio mudarem ou surgir um defeito imprevisto, quão fácil será para os programadores modificar o código sem quebrar partes adjacentes? Por sua vez, a portabilidade avalia a flexibilidade do produto em operar com naturalidade em diferentes sistemas operativos, dispositivos ou configurações de hardware. Em conjunto, essas oito dimensões demonstram que a qualidade de um sistema não reside em ter apenas uma interface agradável ou uma execução rápida, mas sim no equilíbrio harmonioso de toda a sua estrutura técnica e operacional.

Testes de Software na Garantia da Qualidade

Se as normas e os modelos teóricos, como a ISO/IEC 25010, definem o que deve ser alcançado em termos de qualidade, o teste de software é o instrumento que responde se essa meta foi de facto atingida. Ele representa a ponte essencial entre a expectativa descrita nos documentos e a realidade em funcionamento. Longe de ser apenas uma verificação de última hora antes do lançamento, testar constitui um processo investigativo rigoroso e contínuo, focado em submeter o código a condições reais de stress para revelar defeitos antes que eles atinjam o utilizador final (Myers et al., 2012).

Para compreender a sua importância prática, é crucial distinguir a intenção por detrás do teste. Ao contrário da atividade de desenvolvimento, cujo objetivo é construir funcionalidades para que o sistema funcione, o teste adota uma postura crítica e cética: o seu propósito é encontrar onde, como e por que o sistema pode falhar (Pressman & Maxim, 2020). Essa mudança de perspetiva é vital. Nenhum engenheiro consegue provar que um software está 100% livre de falhas; o que os testes estruturados demonstram é a presença de erros e, à medida que estes são corrigidos, a probabilidade de colapso operacional diminui drasticamente, elevando a confiança técnica na solução.

A verificação prática das dimensões da qualidade ocorre em níveis progressivos, refletindo a própria estrutura em camadas de uma aplicação:

- **Testes Unitários:** Operam no nível mais elementar, examinando funções, métodos ou classes de código de forma isolada para atestar a correção algorítmica e a lógica pura (Sommerville, 2019).
- **Testes de Integração:** Avaliam a comunicação entre os diferentes módulos já validados, garantindo que a troca de dados por meio de interfaces e APIs decorra de forma harmoniosa.
- **Testes de Sistema:** Avaliam o ecossistema completo como o software, bases de dados, servidores e redes para validar se o comportamento global respeita os requisitos funcionais e não funcionais estabelecidos.
- **Testes de Aceitação:** Representam a etapa final de homologação, na qual os utilizadores ou clientes validam se o produto satisfaz efetivamente as necessidades operacionais e as regras de negócio do dia a dia (Myers et al., 2012).

Em suma, o teste de software transforma a qualidade num ativo mensurável e tangível. Em vez de supor que uma aplicação é segura, rápida e fiável, a engenharia utiliza casos de teste estruturados para gerar evidências empíricas de conformidade. Testar não é um custo redundante nem uma etapa burocrática; é a salvaguarda operacional que assegura que o sistema entregue ao mundo real seja estável, previsível e digno de confiança humana.

Estudo de Caso: Testes e Qualidade no Contexto Angolano

Para consolidar a articulação entre as formulações teóricas e a prática da engenharia, convém examinar um cenário representativo do ecossistema digital angolano: uma plataforma web de comércio eletrónico dotada de integração com o serviço de pagamentos interbancários Multicaixa Express (MCX). A análise desse ambiente elucida a separação empírica entre falhas de software e falhas sistémicas, ao mesmo tempo que evidencia a necessidade de modelos de qualidade e rotinas rigorosas de teste.

No nível do produto de software, consideram-se os defeitos puramente lógicos introduzidos no código-fonte. Caso o formulário de finalização da compra não valide o formato padronizado dos contactos telefónicos nacionais (nove dígitos iniciados pelo prefixo 9) ou execute de forma incorreta o cálculo do Imposto sobre o Valor Acrescentado (IVA), o sistema incorre numa inconformidade de adequação funcional segundo a ISO/IEC 25010 (ISO/IEC, 2011). Tal anomalia decorre unicamente de falhas na implementação algorítmica ou na verificação sintática, cabendo aos testes unitários a sua identificação antecipada por meio da análise de limites e partições de equivalência (Pressman & Maxim, 2020).

Por outro lado, as vulnerabilidades no nível do sistema computacional transcendem a correção lógica do programa. No momento em que a aplicação efetua a chamada remota à interface de programação de aplicações (API) da rede interbancária para confirmar uma transação, fatores de infraestrutura entram em operação. Flutuações na latência das redes de telecomunicações móveis locais, atrasos no envio de notificações *push* para o telemóvel do utilizador ou instabilidades momentâneas no serviço bancário externo podem impedir que a resposta chegue a tempo ao servidor da loja virtual (Sommerville, 2019). Nessa circunstância, se o sistema não possuir rotinas assíncronas e mecanismos de reconciliação de estado, poderá ocorrer o débito bancário sem a subsequente confirmação do pedido, caracterizando uma falha de fiabilidade e integridade sistémica, e não um erro intrínseco de código.

Para mitigar esses riscos operacionais, o ciclo de testes deve ser executado de maneira escalonada. Inicialmente, realizam-se testes de integração recorrendo a serviços simulados (*mocks*) da API financeira para averiguar como o *backend* gere cenários de sucesso, saldo insuficiente ou tempo limite esgotado (*timeout*). Posteriormente, os testes de sistema e de carga simulam acessos concorrentes em larga escala (condição típica em períodos promocionais ou picos de tráfego de fim de semana), atestando a eficiência de desempenho do servidor e a estabilidade das bases de dados (Myers et al., 2012). Por fim, os testes de aceitação asseguram que a usabilidade da interface seja intuitiva e acessível aos utilizadores locais, confirmando que a solução final entrega valor económico seguro e confiável.

Conclusão

O presente trabalho investigativo analisou a qualidade de sistemas e software, estabelecendo uma trajetória analítica que partiu dos fundamentos etimológicos até à aplicação empírica de padrões e testes no domínio da engenharia. Ao longo do estudo, evidenciou-se que a noção de qualidade transcendeu a visão fabril clássica focada na correção posterior de peças defeituosas, consolidando-se como uma cultura contínua de prevenção de anomalias lógicas durante todas as fases do ciclo de vida computacional.

A distinção conceitual entre sistema computacional e produto de software revelou-se determinante para a maturidade técnica da discussão. Enquanto o software diz respeito ao conjunto lógico de algoritmos, códigos e documentações, o sistema abrange o ecossistema holístico formado pela infraestrutura de hardware, redes, dados e intervenientes humanos. Reconhecer essa fronteira permitiu constatar que a excelência de uma solução tecnológica não se limita à solidez sintática de um programa isolado, mas depende da sua integração estável e harmoniosa no ambiente operacional em que está inserida.

Nesse cenário, modelos consolidados pela indústria, com destaque para a norma ISO/IEC 25010, fornecem a métrica essencial para decompor a qualidade em características objetivas e mensuráveis, tais como adequação funcional, fiabilidade, segurança e

manutenibilidade. Contudo, como demonstrado na fundamentação teórica, tais atributos permanecem apenas no plano abstrato sem a existência de instrumentos práticos de aferição. É precisamente nessa lacuna que o teste de software se estabelece como pilar insubstituível da garantia da qualidade, atuando como o método empírico de verificação e validação capaz de expor vulnerabilidades precocemente e reduzir os custos exponenciais de falhas em produção.

Em síntese, construir sistemas com elevados padrões de qualidade é um compromisso indissociável de rigor metodológico e postura crítica contínua. Para trabalhos futuros, recomenda-se o aprofundamento prático na automação de testes contínuos integrados a pipelines de desenvolvimento, bem como a avaliação do impacto de arquiteturas modernas em nuvem na fiabilidade sistémica. Garantir a qualidade, em última análise, significa entregar soluções computacionais previsíveis, resilientes e plenamente orientadas a servir as necessidades humanas com segurança e eficiência.

Referências

Dicionário Priberam da Língua Portuguesa. (s.d.-a). Qualidade. https://dicionario.priberam.org/qualidade

Dicionário Priberam da Língua Portuguesa. (s.d.-b). Sistema. https://dicionario.priberam.org/sistema

Garvin, D. A. (1988). *Managing quality: The strategic and competitive edge*. Free Press.

ISO/IEC. (2011). *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models* (ISO/IEC Standard No. 25010:2011). https://www.iso.org/standard/35733.html

Myers, G. J., Sandler, C., & Badgett, T. (2012). *The art of software testing* (3.ª ed.). John Wiley & Sons.

Pressman, R. S., & Maxim, B. R. (2020). *Engenharia de software: Uma abordagem profissional* (9.ª ed.). McGraw-Hill.

Sommerville, I. (2019). *Engenharia de software* (10.ª ed.). Pearson.